RICARDO RUIZ

NEOCORE COMPUTER LAB — SIMULAÇÃO E ANÁLISE DA ARQUITETURA INTERNA DE UM COMPUTADOR

Prova de Conceito do Simulador Didático de Arquitetura de Computadores

Relatório técnico apresentado ao curso de **Análise e Desenvolvimento de Sistemas** da **UNIFECAF**, como requisito parcial para obtenção de nota na disciplina de **Computer Architecture and Organization**.

Cliente: NeoCore Tech
Orientador / Tutor: Prof. Vitor Vieira

CAXIAS DO SUL
2026

PROJETO NEOCORE COMPUTER LAB

Prova de Conceito do Simulador Didático de Arquitetura de Computadores

Disciplina: Computer Architecture and Organization	**Cliente:** NeoCore Tech
Função: Analista de Arquitetura de Computadores	**Entregável:** Parte Teórica (25% da Pontuação)

SUMÁRIO

1. Contextualização e Diagnóstico Pedagógico — Seção 1
2. Escolha e Justificativa da Arquitetura — Seção 2
3. Diagrama Geral da Arquitetura — Seção 3
4. Especificação dos Componentes da CPU — Seção 4
5. Conjunto de Instruções (ISA) — Seção 5
6. Subsistema de Memória e Hierarquia — Seção 6
7. Barramentos e Entrada/Saída — Seção 7
8. Ciclo de Instrução (Fetch-Decode-Execute) — Seção 8
9. Estudo de Caso Prático: ADD R1, R2 — Seção 9
10. Métricas de Desempenho — Seção 10
11. Resposta à Questão Central do Projeto — Seção 11
12. Referências Bibliográficas & 13. Siglas — Seções 12-13

1. CONTEXTUALIZAÇÃO E DIAGNÓSTICO PEDAGÓGICO

No ensino de Ciência da Computação e Engenharia, observa-se uma lacuna recorrente: a desconexão conceitual entre o código-fonte de alto nível e a execução física na CPU. Estudantes frequentemente tratam variáveis, operadores e desvios condicionais como abstrações puramente matemáticas, sem compreender a sinalização elétrica e a movimentação de dados registrador-memória que sustentam esses conceitos.

A solução desenvolvida para a NeoCore Tech aborda esse problema por meio de um **Simulador Didático com Transparência de Barramento e Estado**. O objetivo principal é desmistificar o ciclo de máquina, permitindo que o aluno visualize a transição de estados dos registradores, as linhas de controle ativas e a propagação de bits pelos barramentos durante a execução de cada instrução.

1.1 Diagnóstico Pedagógico

Pesquisas em educação em computação (**Patterson & Hennessy, 2014**; **Tanenbaum, 2013**) indicam que alunos apresentam dificuldades específicas em:

- **Visualizar o ciclo de máquina** como sequência temporal de micro-operações;
- **Distinguir dados de instruções** quando compartilham o mesmo espaço de memória (Von Neumann);
- **Compreender a função dos barramentos** como vias físicas de comunicação;
- **Relacionar registradores, cache e RAM** dentro da hierarquia de memória.

O simulador proposto ataca cada uma dessas lacunas por meio de representação visual explícita de estados e sinais.

2. ESCOLHA E JUSTIFICATIVA DA ARQUITETURA

A arquitetura projetada adota o **Modelo de Von Neumann** como base estrutural, integrando princípios de **Organização RISC** (Reduced Instruction Set Computer) e **Arquitetura Load/Store**.

2.1 Justificativas Teóricas e Pedagógicas

Unicidade de Memória (Von Neumann): Dados e instruções compartilham o mesmo espaço de endereçamento. Isso permite ensinar conceitos vitais como carregamento de programas na RAM, alteração de ponteiros de memória e a distinção prática entre opcode e operando. A simplicidade do modelo também facilita a visualização do "gargalo de Von Neumann" — a limitação de um único barramento compartilhado entre instruções e dados.

Conjunto Reduzido de Instruções (RISC): Instruções com tamanho fixo (16 bits) e formatos simplificados reduzem a complexidade da Unidade de Controle. Isso torna previsível a contagem de ciclos de clock por instrução (CPI) e facilita a decodificação visual.

Arquitetura Load/Store: As operações aritméticas ocorrem exclusivamente entre registradores internos da CPU. O acesso à memória é isolado em instruções dedicadas (LOAD e STORE), facilitando a separação entre busca e execução.

2.2 Comparação com Arquiteturas Reais

Característica	NeoCore (didática)	MIPS32	ARM Cortex-M	x86 (CISC)
Largura de palavra	16 bits	32 bits	32 bits	32/64 bits
Tamanho da instrução	Fixo (16 bits)	Fixo (32 bits)	Variável (16/32)	Variável (1–15 bytes)
Modo de organização	Load/Store	Load/Store	Load/Store	Registrador-Memória
Nº de registradores	4 (R0–R3)	32	16	8 (legado)
Unidade de Controle	Hardwired	Hardwired	Hardwired	Microprogramada
Complexidade didática	Baixa	Média	Média	Alta

Justificativa da simplificação: A redução para 4 registradores e 16 bits mantém a integridade conceitual (Load/Store, RISC, barramentos separados) sem sobrecarregar o aluno com detalhes irrelevantes no aprendizado inicial.

2.3 Por que não pipeline?

Optou-se por **não implementar pipeline** nesta prova de conceito. Justificativa: o pipeline introduz dependências de dados e hazards que desviam o foco didático do ciclo básico fetch-decode-execute. Uma versão futura (v2.0) poderá incluir pipeline de 3 estágios como módulo avançado.

3. DIAGRAMA GERAL DA ARQUITETURA

A Figura 1 apresenta o diagrama geral da arquitetura NeoCore, evidenciando a CPU (com ULA, Unidade de Controle e Banco de Registradores), os três barramentos de comunicação, a memória cache L1, a memória principal RAM e os dispositivos de entrada e saída. As setas indicam a direcionalidade do fluxo de dados, endereços e sinais de controle entre os subsistemas.



FIGURA 1 — Diagrama Geral da Arquitetura NeoCore

Legenda da Figura 1:

- **Linhas espessas / duplas:** Barramento de Dados (16 bits, bidirecional)
- **Linhas tracejadas / simples:** Barramento de Endereços (12 bits, unidirecional CPU→Memória)
- **Linhas pontilhadas:** Barramento de Controle (sinais individuais)
- As setas indicam a direcionalidade do fluxo em cada barramento.

Observações sobre o diagrama:

1. A CPU é o único mestre do barramento de endereços (unidirecional).
2. O barramento de dados é compartilhado entre CPU, cache, RAM e E/S (bidirecional).
3. A Unidade de Controle emite sinais para todos os componentes via barramento de controle.
4. A cache L1 fica entre a CPU e a RAM, reduzindo o tempo médio de acesso.

4. ESPECIFICAÇÃO DOS COMPONENTES DA CPU

4.1 Unidade Lógica e Aritmética (ULA / ALU)

A ULA opera sobre palavras de **16 bits** em complemento de dois, realizando operações aritméticas (ADD, SUB) e lógicas (AND, OR, NOT, XOR). Possui registrador de status interno (**Flags Register**):

Flag	Nome	Descrição
Z	Zero	Ativado quando o resultado da operação é 0
N	Negative	Ativado quando o bit mais significativo (MSB) do resultado é 1
C	Carry	Indica estouro de capacidade em somas sem sinal
O	Overflow	Indica estouro em somas com sinal em complemento de dois

Tabela de operações da ULA:

ALUOp (4 bits)	Operação	Descrição
0000	AND	A AND B (bit a bit)
0001	OR	A OR B
0010	XOR	A XOR B
0011	NOT	NOT A (B ignorado)
0100	ADD	A + B
0101	SUB	A − B

ALUOp (4 bits)	Operação	Descrição
0110	SHL	A deslocado 1 bit à esquerda
0111	SHR	A deslocado 1 bit à direita

4.2 Unidade de Controle (UC)

Projetada no modelo **Hardwired (Decodificada por Matriz Controladora)** para demonstrar o sincronismo por clock de máquina. A UC recebe o Opcode vindo do Instruction Register (IR) e gera a sequência exata de pulsos de controle a cada borda de subida do clock.

Matriz de Controle (extrato):

Opcode (4 bits)	Instrução	Sinais de Controle Ativados
0000	NOP	(nenhum)
0001	ADD Rd, Rs	RegRead, ALUOp=0100, RegWrite
0010	SUB Rd, Rs	RegRead, ALUOp=0101, RegWrite
0011	AND Rd, Rs	RegRead, ALUOp=0000, RegWrite
0100	OR Rd, Rs	RegRead, ALUOp=0001, RegWrite
0101	NOT Rd	RegRead, ALUOp=0011, RegWrite
0110	LOAD Rd, addr	MEM_READ, MAR_in, MBR_out, RegWrite
0111	STORE Rs, addr	MEM_WRITE, MAR_in, MBR_in
1000	JMP addr	PC_in
1001	JZ addr	PC_in (se Z=1)
1111	HLT	HALT (para o clock)

Sinais de controle individuais gerados pela UC:

Sinal	Função
PC_out / PC_in	Habilita leitura/escrita no Program Counter
MAR_in	Carrega o Memory Address Register
MBR_out / MBR_in	Habilita leitura/escrita no Memory Buffer Register
IR_in	Carrega o Instruction Register
RegRead / RegWrite	Habilita leitura/escrita no banco de registradores
ALUOp[3:0]	Seleciona a operação da ULA
FlagWrite	Atualiza o registrador de flags
MEM_READ / MEM_WRITE	Ativa leitura/escrita na memória
IO_SELECT	Diferencia acesso à RAM de acesso a E/S
HALT	Para o clock

4.3 Banco de Registradores

Registrador	Largura	Função
PC (Program Counter)	12 bits	Guarda o endereço da próxima instrução
IR (Instruction Register)	16 bits	Armazena a instrução recém-buscada
MAR (Memory Address Register)	12 bits	Mantém o endereço a ser lido/escrito
MBR/MDR (Memory Buffer Register)	16 bits	Armazena dado lido/escrito da memória
R0, R1, R2, R3	16 bits cada	Registradores de uso geral
Flags (Z, N, C, O)	4 bits	Estado da última operação da ULA

Total: 4 registradores visíveis ao programador (R0–R3) + 5 registradores internos (PC, IR, MAR, MBR, Flags).

5. CONJUNTO DE INSTRUÇÕES (ISA)

5.1 Formatos de Instrução (16 bits)

Formato R (Registrador-Registrador): `Opcode (15-12) | RD (11-10) | RS (9-8) | Reservado (7-0)`

Formato I (Imediato / Memória): `Opcode (15-12) | RD/RS (11-10) | Endereço/Imediato (9-0)`

Formato J (Jump Incondicional): `Opcode (15-12) | Endereço de destino (11-0)`

5.2 Tabela Completa de Instruções

Mnemônico	Opcode	Formato	Operação	Flags afetadas
NOP	0000	—	Nenhuma operação	—
ADD Rd, Rs	0001	R	Rd ← Rd + Rs	Z, N, C, O
SUB Rd, Rs	0010	R	Rd ← Rd − Rs	Z, N, C, O
AND Rd, Rs	0011	R	Rd ← Rd AND Rs	Z, N
OR Rd, Rs	0100	R	Rd ← Rd OR Rs	Z, N
NOT Rd	0101	R	Rd ← NOT Rd	Z, N
LOAD Rd, addr	0110	I	Rd ← RAM[addr]	—
STORE Rs, addr	0111	I	RAM[addr] ← Rs	—
JMP addr	1000	J	PC ← addr	—
JZ addr	1001	J	Se Z=1: PC ← addr	—
HLT	1111	—	Para o clock	—

5.3 Modos de Endereçamento e Codificação

Suporta endereçamento **por registrador** (ex.: `ADD R1, R2`), **direto** (ex.: `LOAD R1, 0x100`) e **imediato** (ex.: `LOAD R1, #5`). Adota **big-endian** e complemento de dois de 16 bits (−32.768 a +32.767).

6. SUBSISTEMA DE MEMÓRIA E HIERARQUIA

6.1 Hierarquia de Memória

Nível	Tempo Médio de Acesso	Capacidade Didática	Tecnologia Simulada
Registradores	1 ciclo de clock	8 palavras (16 bits)	Flip-Flops / Latches
Cache L1	2–3 ciclos de clock	64 palavras	SRAM (Mapeamento Direto)
Memória Principal (RAM)	20–50 ciclos de clock	4.096 palavras (4K)	DRAM
Armazenamento / E/S	> 500 ciclos de clock	Bloco de simulação de disco	Dispositivo de Bloco NVM

6.2 Funcionamento da Cache L1

Operação por **Mapeamento Direto** com política **Write-Through** (escrita simultânea em cache e RAM).
Endereço de 12 bits: **Tag** (bits 11–6, 6 bits), **Index** (bits 5–0, 6 bits). **Offset** = 0 (1 palavra por bloco).

- Exemplo: endereço 0x0A5 (binário `0000 1010 0101`) → **Tag** = 0x02, **Index** = 0x25 (linha 37).
- **Hit:** dado retornado em 2 ciclos.
- **Miss:** bloco transferido da RAM (20–50 ciclos) para a linha 37, depois entregue.

6.3 Mapa de Memória

Faixa de Endereço	Uso
0x000 – 0x7FF	Programa (instruções)
0x800 – 0xEFF	Dados (variáveis, pilha)
0xF00 – 0xF7F	E/S mapeada (registradores de dispositivo)
0xF80 – 0xFFF	Reservado / vetores de reset

7. BARRAMENTOS E DISPOSITIVOS DE ENTRADA/SAÍDA

7.1 Estrutura dos Barramentos

A comunicação entre CPU, Memória e Dispositivos de E/S é realizada por **três barramentos principais**, cada um com função específica:

- **Barramento de Dados (Data Bus):** Bidirecional, com largura de **16 bits**. Trafega tanto instruções (durante o ciclo de fetch) quanto dados (durante operações de LOAD/STORE e E/S) entre a CPU, a memória cache, a RAM e os periféricos. Sua bidirecionalidade é essencial: no fetch, a memória "empurra" a instrução para a CPU; no store, a CPU "empurra" o dado para a memória.
- **Barramento de Endereços (Address Bus):** Unidirecional (CPU → Memória/Periféricos), com largura de **12 bits**. Apenas a CPU pode originar um endereço, o que evita conflitos de barramento. Com 12 bits, o espaço de endereçamento é de 2^{12} = **4.096 posições** (4K palavras), compatível com a RAM didática de 4K × 16.
- **Barramento de Controle (Control Bus):** Conjunto de linhas individuais que sincronizam e comandam as operações. Os sinais são:

Sinal	Função
MEM_READ	Habilita leitura da memória (RAM ou cache)
MEM_WRITE	Habilita escrita na memória
IO_SELECT	Diferencia acesso à RAM de acesso a E/S
CLOCK	Sincroniza a transição de estado de todos os componentes
RESET	Reinicia registradores e invalida cache
HALT	Para o clock (fim de programa)

7.2 Dispositivos de E/S

Dispositivo	Endereço	Tipo	Função Didática
Teclado	0xF00	Entrada	Simula leitura de caractere (código ASCII)
Display	0xF01	Saída	Exibe valor de 16 bits em hexadecimal
Disco simulado	0xF10 – 0xF1F	Bloco	Simula setor de disco para persistência

7.3 Mapeamento Memory-Mapped I/O, Boot e Exceções

O simulador adota **I/O mapeada em memória**: os registradores dos dispositivos ocupam endereços no espaço de 12 bits. Quando a UC ativa `IO_SELECT` e o endereço está na faixa 0xF00–0xF7F, o acesso é redirecionado ao dispositivo. As mesmas instruções `LOAD`/`STORE` servem tanto para RAM quanto para E/S — simplificação pedagógica alinhada aos modelos ARM e MIPS.

Fluxo de uma escrita no Display (exemplo):

1. CPU executa `STORE R1, 0xF01`
2. `MAR ← 0xF01`
3. UC ativa `IO_SELECT + MEM_WRITE`
4. Barramento de dados transporta R1 → display
5. Display atualiza sua saída

Sequência de Reset/Boot: `PC ← 0x000`; registradores `IR, MAR, MBR, R0–R3, Flags` são zerados; cache marcada como inválida; UC passa ao estado FETCH.

Instrução Inválida: Se o opcode não corresponde a nenhuma instrução da ISA, a UC ativa `HALT`, a flag de erro E é setada, o PC congela no endereço da instrução e o display exibe 0xEEEE como código de erro.

8. CICLO DE INSTRUÇÃO (FETCH-DECODE-EXECUTE)

8.1 Fases e Tabela de Micro-operações

Etapa	Micro-operação	Sinais de Controle
T0	MAR ← PC	PC_out, MAR_in
T1	MBR ← RAM[MAR]; PC ← PC+1	MEM_READ, PC_inc
T2	IR ← MBR	MBR_out, IR_in
T3	UC decodifica opcode	— (interno)
T4	ULA ← R1 op R2	RegRead, ALUOp
T5	R1 ← ULA_out; Flags ← ULA	RegWrite, FlagWrite

8.2 Diagrama de Temporização

A Figura 2 ilustra a sequência temporal dos sinais de controle ao longo dos seis pulsos de clock (T0–T5) durante a execução da instrução `ADD R1, R2`, considerando o estado inicial descrito na Seção 9.



FIGURA 2 — Diagrama de Temporização do ciclo ADD R1, R2

Leitura do diagrama:

- **T0:** PC (0x00A) é transferido ao MAR.
- **T1:** `MEM_READ` é ativado; a RAM coloca 0x1600 no MBR; o PC incrementa para 0x00B.
- **T2:** O conteúdo do MBR é copiado para o IR.
- **T3:** A UC decodifica o opcode 0001 (ADD) e configura os registradores de origem/destino.
- **T4:** A ULA calcula `0x0005 + 0x0003`, disponibilizando 0x0008 em sua saída.
- **T5:** `RegWrite` é ativado; o resultado 0x0008 é gravado em R1 e as Flags (Z=0, N=0, C=0, O=0) são atualizadas.

9. ESTUDO DE CASO PRÁTICO: ADD R1, R2

Instrução: ADD R1, R2 → Opcode 0001 | RD=01 | RS=10 | Unused=00000000 ⇒ **Codificação 0x1600**
Estado inicial: PC = 0x00A, R1 = 0x0005, R2 = 0x0003, RAM[0x00A] = 0x1600

Passo	Fase	Ação	Estado após
1	Fetch T0	MAR ← PC	MAR = 0x00A
2	Fetch T1	MBR ← RAM[0x00A]; PC ← PC+1	MBR = 0x1600; PC = 0x00B
3	Fetch T2	IR ← MBR	IR = 0x1600
4	Decode T3	UC lê opcode 0001 (ADD), RD=R1, RS=R2	Entradas ULA: A=0x0005, B=0x0003
5	Execute T4	ULA calcula 0x0005 + 0x0003	ULA_out = 0x0008
6	Execute T5	R1 ← ULA_out; Flags ← ULA	R1 = 0x0008; Z=0, N=0, C=0, O=0

Resultado final: R1 = 0x0008 e PC = 0x00B, pronto para a instrução seguinte.

10. MÉTRICAS DE DESEMPENHO

10.1 CPI e Impacto da Cache

Classe	Instruções	CPI	Observação
R-type (ULA)	ADD, SUB, AND, OR, NOT	6	3 fetch + 1 decode + 2 execute
I-type (LOAD)	LOAD	7	+1 ciclo de acesso à memória
I-type (STORE)	STORE	7	+1 ciclo de escrita
J-type	JMP, JZ	5	Sem fase de execução na ULA
HLT	HLT	2	Interrompe o ciclo

Speedup da Cache:
• Sem cache = 50 ciclos (acesso à RAM).
• Com cache (Hit 90% a 3 ciclos / Miss 10% a 50 ciclos) = 7,7 ciclos médios.
• Speedup ≈ 50 / 7,7 ≈ **6,5×**.
Frequência Didática: Operação padrão em **10 Hz** (100 ms/ciclo) para observação passo a passo. Modo "turbo" a 1 kHz para execução contínua.

11. RESPOSTA À QUESTÃO CENTRAL DO PROJETO

> ***"Como os diferentes componentes de um computador trabalham de forma coordenada para buscar, interpretar e executar uma instrução?"***

A coordenação entre os componentes é estabelecida pela **sincronização por clock** e pela **sinalização da Unidade de Controle** sobre os barramentos. Nenhum componente atua de forma isolada:

1. O **Program Counter (PC)** define o endereço da instrução a ser buscada.
2. Os **Barramentos de Endereço, Dados e Controle** transportam a palavra da memória/cache para o **Instruction Register (IR)**.
3. A **Unidade de Controle (UC)** decodifica o opcode e emite os sinais elétricos que comutam portas lógicas e barramentos nos instantes exatos.
4. A **Unidade Lógica e Aritmética (ULA)** executa as operações com os operandos fornecidos pelo **Banco de Registradores**, atualizando os sinalizadores (Flags).
5. O resultado retorna aos **registradores, memória ou dispositivos de E/S**.

Essa sequência contínua de movimentação de dados e alteração de estados lógicos transforma linhas abstratas de código em ações computacionais concretas. O simulador NeoCore torna cada uma dessas etapas **visível, temporal e verificável**, fechando a lacuna pedagógica identificada na Seção 1.

12. REFERÊNCIAS BIBLIOGRÁFICAS

- TANENBAUM, Andrew S.; BOS, Herbert. **Organização Estruturada de Computadores**. 6. ed. São Paulo: Pearson, 2013.
- STALLINGS, William. **Arquitetura e Organização de Computadores**. 10. ed. São Paulo: Pearson, 2017.
- PATTERSON, David A.; HENNESSY, John L. **Computer Organization and Design: The Hardware/Software Interface**. 5. ed. San Francisco: Morgan Kaufmann, 2014.
- HENNESSY, John L.; PATTERSON, David A. **Computer Architecture: A Quantitative Approach**. 6. ed. San Francisco: Morgan Kaufmann, 2019.
- LOGISIM EVOLUTION. **Manual da Ferramenta Logisim Evolution**. Disponível em: <https://github.com/logisim-evolution/logisim-evolution>. Acesso em: 8 nov. 2026.
- TINKERCAD. **Tinkercad Circuits — Learning Center**. Disponível em: <https://www.tinkercad.com/learn/circuits>. Acesso em: 8 nov. 2026.

13. GLOSSÁRIO DE SIGLAS

Sigla	Significado	Sigla	Significado
ALU / ULA	Arithmetic Logic Unit / Unidade Lógica e Aritmética	IR	Instruction Register
CPI	Cycles Per Instruction	MAR	Memory Address Register
CPU	Central Processing Unit	MBR / MDR	Memory Buffer / Data Register
DRAM / SRAM	Dynamic / Static Random Access Memory	PC	Program Counter
E/S (I/O)	Entrada/Saída (Input/Output)	RISC	Reduced Instruction Set Computer
ISA	Instruction Set Architecture	UC	Unidade de Controle